



BMCS2013 Data Engineering

CLO2 ASSIGNMENT 202409

Assignment Title : **Malay-English Lexicon: POS, Word Sense, Sentiment, Synonym, Anonym**

Team Reference : **G4-5**

Programme / Group : **RDS2 Group4**

Student Name	Student ID	Contribution	Signature
CASON SOH KAI SHENG	24WMR08849	25 %	<i>Cason</i>
GAN SHE MING	24WMR08864	25%	<i>Ming</i>
HEA HUI MIN	24WMR08865	25 %	<i>Min</i>
ETHEL NG YI YAN	24WMR08861	25 %	<i>Ethel</i>
TOTAL (must sum to 100%)		100%	

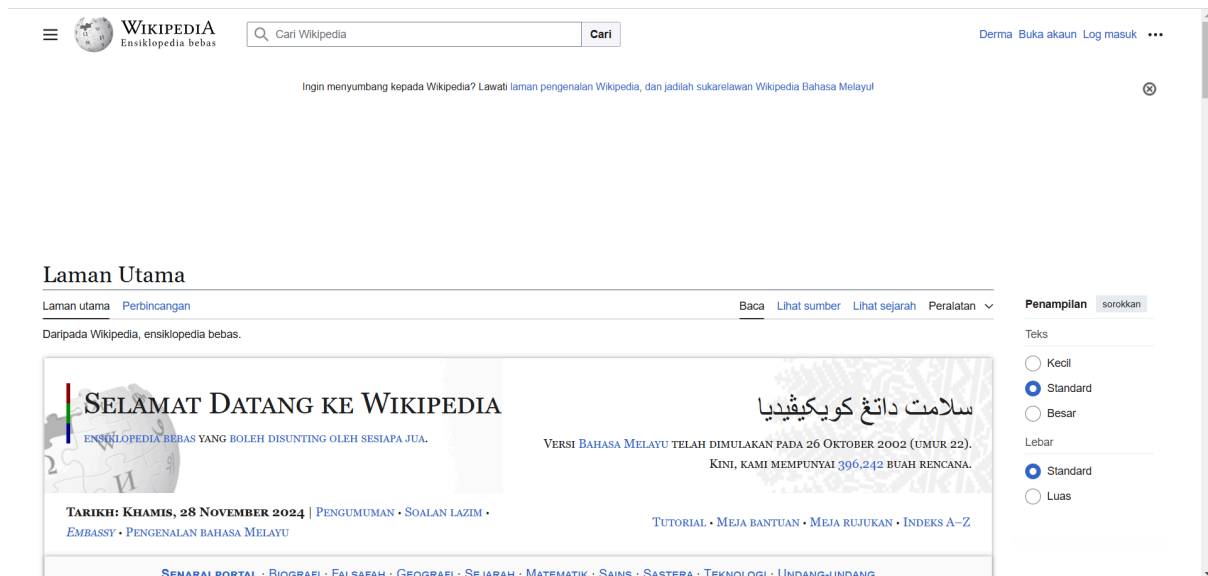
CASON SOH KAI SHENG	GAN SHE MING	HEA HUI MIN	ETHEL NG YI YAN
			

1. Data Collection and Preparation

1.1. Data Source(s)

1.1.1 Data source 1: Wikipedia

- URL: <https://ms.wikipedia.org>

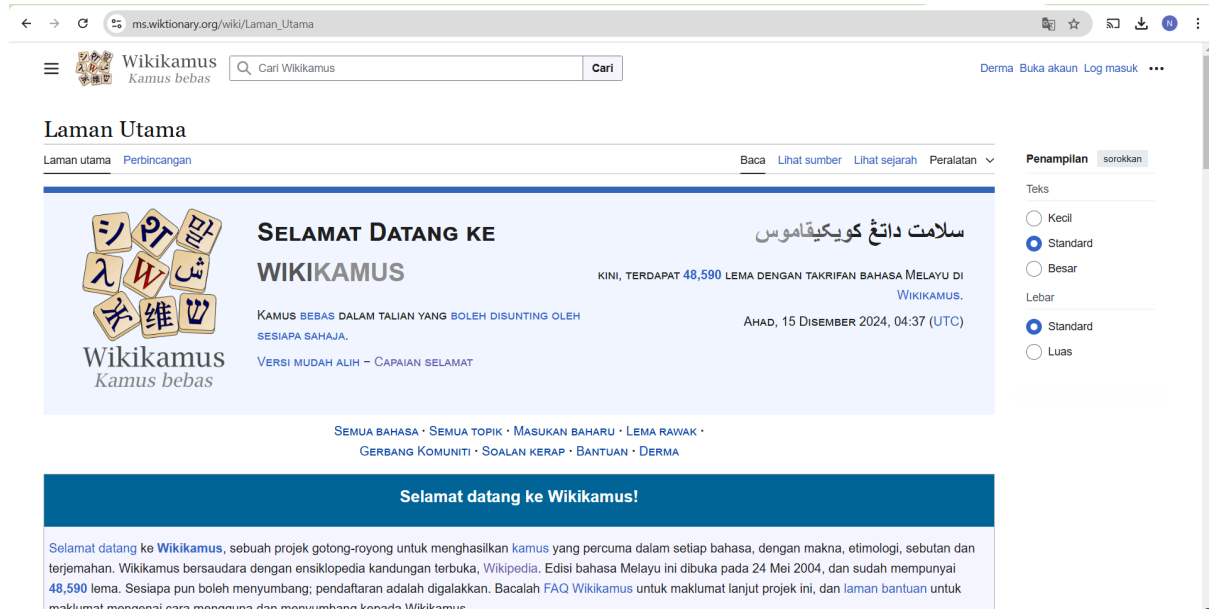


Our project's primary data source would be the Malay Wikipedia (<https://ms.wikipedia.org>). We will be using the method web scraping to extract all the words including Malay and English from various Malay articles and proceed to clean it. The extracted data will be used to build the lexicon

- Python libraries/API used: <https://ms.wikipedia.org/wiki/>

1.1.2 Data source 1: Wikipedia

- Data Source 2: Wiktionary
- URL : https://ms.wiktionary.org/wiki/Laman_Utama



Wiktionary will be our source to enrich our lexicon. We will be using the method, data crawling to extract the related data and use them to assign each word with their respective POS (part-of-speech) and word sense.

- Python libraries/API used: <https://ms.wiktionary.org/w/api.php>

1.2. List of Python classes and Jupyter notebook(s) for data collection, cleaning and preprocessing:

Name of Python class(es) / notebooks	Author
hdfs_handler.py	Gan She Ming
web_scraper.py	Gan She Ming
json_extractor.py	Ethel Ng Yi Yan
text_cleaner.py	Ethel Ng Yi Yan
redis_replacer.py	Ethel Ng Yi Yan
tokenizer.py	Ethel Ng Yi Yan
data_processing.py	Gan She Ming

api_crawler.py	Gan She Ming
pos_extractor.py	Gan She Ming
interactive_tool.py	Gan She Ming
fill_missing.py	Gan She Ming
modify_column.py	Gan She Ming
data_options.py	Gan She Ming
lexicon_functions.py	Gan She Ming

1.3. Code for Python Classes

1.3.1 hdfs_handler.py

```
import os
import subprocess

class HDFSHandler:
    @staticmethod
    def upload_to_hdfs(local_path, hdfs_path):
        try:
            # Create the directory in HDFS if it doesn't exist
            subprocess.run(
                ["hadoop", "fs", "-mkdir", "-p",
                os.path.dirname(hdfs_path)],
                check=True,
                capture_output=True,
            )
            # Upload the file to HDFS
            subprocess.run(
                ["hadoop", "fs", "-put", "-f", local_path,
                hdfs_path],
                check=True,
                capture_output=True,
            )
            print(f"File {local_path} uploaded to HDFS at {hdfs_path}")
            return True
        except subprocess.CalledProcessError as e:
```

```
print(f"HDFS error: {e.stderr.decode()}")
return False
```

1.3.2 web_scraper.py

```
import requests
from bs4 import BeautifulSoup
import json
import os

class WebScraper:
    def __init__(self, output_folder):
        self.output_folder = output_folder
        if not os.path.exists(output_folder):
            os.makedirs(output_folder)

    def fetch_article(self, title):
        try:
            url = "https://ms.wikipedia.org/wiki/" + title
            response = requests.get(url)

            if response.status_code != 200:
                print(f"Failed to fetch {title}: HTTP {response.status_code}")
                return None

            # Parse the HTML content
            soup = BeautifulSoup(response.text, 'html.parser')
            paragraphs = soup.find_all('p')
            merged_paragraph = ' '.join(paragraph.text.strip() for paragraph in paragraphs)

            return {"title": title, "content": merged_paragraph}
        except Exception as e:
            print(f"Error fetching article {title}: {e}")
            return None

    def save_to_file(self, data):
        try:
            json_file_name = f"{data['title']}.json"
            json_file_path = os.path.join(self.output_folder, json_file_name)
```

```

        with open(json_file_path, mode='w', encoding='utf-8')
as file:
            json.dump(data, file, ensure_ascii=False,
indent=4)
            print(f"Data has been saved to {json_file_path}")
            return json_file_path
    except Exception as e:
        print(f"Error saving file for {data['title']}: {e}")
        return None

```

1.3.3 json_extractor.py

```

import os
import json

class JSONExtractor:
    def __init__(self, directory_path, output_file):
        self.directory_path = directory_path
        self.output_file = output_file
        self.json_contents = {}

    def load_json_files(self):
        """Load all JSON files from a directory."""
        for filename in os.listdir(self.directory_path):
            if filename.endswith(".json"):
                file_path = os.path.join(self.directory_path,
filename)
                try:
                    with open(file_path, 'r') as file:
                        self.json_contents[filename] =
json.load(file)
                        print(f"Successfully read content from:
{filename}")
                except Exception as e:
                    print(f"Error reading {filename}: {e}")

    def extract_content(self):
        """Extract 'content' field from loaded JSON files."""
        contents = {}
        for filename, data in self.json_contents.items():
            contents[filename] = data.get("content", "No content
available.")

```

```

        print(f"Successfully extracted content from:
{filename}")

    try:
        with open(self.output_file, 'w') as output_file:
            json.dump(contents, output_file, indent=4)
            print(f"Contents written to: {self.output_file}")
    except Exception as e:
        print(f"Error writing to {self.output_file}: {e}")

```

1.3.4 text_cleaner.py

```

import json
import re

class TextCleaner:
    def __init__(self, stop_words, english_noise_words):
        self.stop_words = stop_words
        self.english_noise_words = english_noise_words

    def clean_text(self, text):
        """Clean text by removing Jawi, Greek words, special
characters, digits, and stop words."""
        text = re.sub(r'[\u0600-\u06FF]+', '', text) # Remove
Jawi
        text = re.sub(r'[\u0370-\u03FF]+', '', text) # Remove
Greek
        text = re.sub(r'^\w\s-', '', text) # Remove
special characters
        text = re.sub(r'\d+', '', text) # Remove
digits
        words = text.split()
        filtered_words = [word.lower() for word in words if
word.lower() not in self.stop_words and word.lower() not in
self.english_noise_words]
        return ' '.join(filtered_words)

    def process_json(self, input_file_path, output_file_path):
        """Clean all text fields in a JSON file."""

```

```

try:
    with open(input_file_path, 'r') as input_file:
        data = json.load(input_file)

    for key in data:
        if isinstance(data[key], str):
            data[key] = self.clean_text(data[key])

    with open(output_file_path, 'w') as output_file:
        json.dump(data, output_file, indent=4)
    print(f"Cleaned data saved to: {output_file_path}")

except Exception as e:
    print(f"Error processing JSON file: {e}")

```

1.3.5 redis_replacer.py

```

import json
import redis
import re

class RedisReplacer:
    def __init__(self, redis_host='localhost', redis_port=6379,
db=0):
        self.redis_client = redis.Redis(host=redis_host,
port=redis_port, db=db)

    def load_replacement_dict(self, replacement_dict):
        """Load a replacement dictionary into Redis."""
        self.redis_client.mset(replacement_dict)

    def replace_words(self, content):
        """Replace words in the content using Redis keys."""
        if isinstance(content, dict):
            return {key: self.replace_words(value) for key, value
in content.items()}
        elif isinstance(content, list):
            return [self.replace_words(item) for item in content]
        elif isinstance(content, str):
            for key in self.redis_client.keys():

```

```

        key_str = key.decode("utf-8")
        value_str =
self.redis_client.get(key).decode("utf-8")
        content = re.sub(rf"\b{key_str}\b", value_str,
content)

        return content
    else:
        return content

def process_json(self, input_file_path, output_file_path):
    """Process a JSON file and replace words using Redis."""
    with open(input_file_path, 'r') as file:
        data = json.load(file)

    updated_data = self.replace_words(data)

    with open(output_file_path, 'w') as file:
        json.dump(updated_data, file, ensure_ascii=False,
indent=4)
    print(f"Replaced content saved to: {output_file_path}")

```

1.3.6 tokenizer.py

```

import json
import csv

class Tokenizer:
    @staticmethod
    def tokenize(text):
        """Tokenize text into words."""
        return text.split()

    def process_json(self, input_file_path, output_csv_path):
        """Tokenize JSON content and save unique words to a CSV
file."""
        try:
            with open(input_file_path, 'r') as input_file:
                data = json.load(input_file)

            all_words = []
            for key in data:

```

```

        if isinstance(data[key], str):
            tokens = self.tokenize(data[key])
            all_words.extend(tokens)

    unique_words = sorted(set(all_words))

    with open(output_csv_path, 'w', newline='') as
csv_file:
        writer = csv.writer(csv_file)
        writer.writerow(["Unique Words"])
        for word in unique_words:
            writer.writerow([word])

    print(f"Unique words saved to: {output_csv_path}")
    print(f"Total unique words: {len(unique_words)}")

except Exception as e:
    print(f"Error processing JSON file: {e}")

```

1.3.7 data_processing.py

```

import csv

class DataProcessing:
    @staticmethod
    def read_words_from_csv(file_path):
        """
        Reads words from the CSV file.
        Expects a column 'Unique Words'.
        """
        words = []
        with open(file_path, mode='r', encoding='utf-8') as file:
            reader = csv.DictReader(file)  # Use DictReader to
access columns by name
            for row in reader:
                word = row.get("Unique Words")  # Access the
'Unique Words' column
                if word:  # Ensure the word value is not empty or
None
                    words.append(word.strip())
        return words

```

```

    @staticmethod
    def save_to_csv(processed_data, output_file):
        with open(output_file, mode='w', encoding='utf-8',
newline='') as file:
            writer = csv.writer(file)
            # Write the header
            writer.writerow(["Word", "Sentiment", "POS", "Word
Senses"])
            # Write the data
            for data in processed_data:
                writer.writerow([
                    data['word'],
                    data['sentiment'],
                    data['pos'],
                    data['sense']
                ])

```

1.3.8 api_crawler.py

```

import requests
import re

class ApiCrawler:
    @staticmethod
    def crawl_data_from_api(word):
        base_url = "https://ms.wiktionary.org/w/api.php"
        params = {
            "action": "query",
            "prop": "extracts", # Request the textual content of
the page
            "format": "json", # Return the output in JSON
format
            "titles": word, # Specify the title of the word
to fetch
        }

        try:
            response = requests.get(base_url, params=params,
timeout=10) # 10-second timeout

```

```

        response.raise_for_status() # Ensure the request was
successful
        data = response.json() # Parse JSON response

        # Extract content from the returned JSON
        pages = data.get("query", {}).get("pages", {})
        for page_id, page_content in pages.items():
            if page_id != "-1": # Check if the page exists
                extract = page_content.get("extract", "") #
Get the page extract
                pos = extract_pos(extract) # Extract POS from
the content

                word_senses =
extract_word_senses_from_text(extract) # Extract word senses
                return pos, word_senses
            return "N/A", [] # Default if no content found
        except requests.exceptions.RequestException as e:
            print(f"Error fetching data for word {word}: {e}")
            return "N/A", []

    @staticmethod
    def extract_pos(content):
        """
        Extract a maximum of 2 POS from content based on sorted
keyword matching.
        Matches exact words to avoid partial matches.
        """
        matched_pos = []
        for pos, keywords in sorted_pos_keywords.items():
            for keyword in keywords:
                # Use regex to match keyword as a whole word
                if re.search(rf'\b{re.escape(keyword)}\b',
content.lower()):
                    matched_pos.append(pos)
                    break # Avoid matching the same POS multiple
times

            if len(matched_pos) >= 2:
                break # Limit to 2 POS matches
        return ";".join(matched_pos) if matched_pos else "N/A"

    @staticmethod

```

```

def extract_word_senses_from_text(html_content):
    """
    Extract word senses from <ol> tags within the HTML content
    using string operations.
    Ensure each word sense is stored on a single line by
    removing newline characters.
    Limit the number of senses to 2.
    """
    word_senses = []

    # Use regex to extract content inside <ol>...</ol>
    ol_matches = re.findall(r'<ol.*?>(.*?)</ol>',
html_content, re.DOTALL)

    for ol_content in ol_matches:
        # Find each <li> within the matched <ol>
        li_matches = re.findall(r'<li.*?>(.*?)</li>',
ol_content, re.DOTALL)

        for li_content in li_matches:
            # Clean up any HTML tags within the <li> content
            and remove newlines
            cleaned_content = re.sub(r'<.*?>', '',
li_content).strip()
            cleaned_content = re.sub(r'[\r\n]+', ' ',
cleaned_content) # Replace newlines with spaces
            if cleaned_content:
                word_senses.append(cleaned_content)

            # Limit to 2 word senses
            if len(word_senses) >= 2:
                break
        if len(word_senses) >= 2:
            break

    return word_senses

```

1.3.9 pos_extractor.py

```

class PosExtractor:
    @staticmethod
    def sorted_pos_keywords():

```

```

pos_keywords = {
    "kata nama": ["nama", "kata nama", "noun"], # Nouns
    "kata kerja": ["kerja", "kata kerja", "verb"], #
Verbs
    "kata adjektif": ["adjektif", "kata sifat",
"adjective"], # Adjectives
    "kata ganti nama": ["ganti nama", "pronoun"], #
Pronouns
    "kata bilangan": ["bilangan", "numeral"], # Numerals
    "kata adverba": ["adverba", "adverb"], # Adverbs
    "kata seru": ["seru", "interjection"], #
Interjections
    "kata hubung": ["hubung", "conjunction"], #
Conjunctions
    "kata sendi nama": ["sendi nama", "preposition"], #
Prepositions
    "kata nafi": ["nafi", "negation"], # Negators
    "kata tanya": ["tanya", "question"], # Question Words
    "kata perintah": ["perintah", "imperative"], #
Imperatives
}

# Sort POS keywords by the length of their keywords
(longest first)
return dict(
    sorted(
        pos_keywords.items(),
        key=lambda item: max(len(keyword) for keyword in
item[1]),
        reverse=True
    )
)

```

13.10 interactive_tool

```

from fill_missing import FillMissing
from modify_column import ModifyColumn

class InteractiveTool:
    @staticmethod
    def run_tool(input_file, output_file):
        print("Choose an option:\n1. Fill missing values\n2.

```

```

Modify specific column data")
    choice = input("Enter 1 or 2: ").strip()

    if choice == "1":
        FillMissing.detect_and_fill_missing_values(input_file,
output_file)
    elif choice == "2":
        ModifyColumn.modify_column_for_word(input_file,
output_file)
    else:
        print("Invalid choice. Exiting tool.")

```

1.3.11 fill_missing.py

```

import csv
from data_options import DataOptions

class FillMissing:
    @staticmethod
    def detect_and_fill_missing_values(input_file, output_file):
        """
        Detects missing values in the CSV and interactively
prompts the user to fill them.
Progress is saved after every row is processed.
        """
        with open(input_file, mode='r', encoding='utf-8') as file:
            reader = csv.DictReader(file)
            rows = list(reader)

            fieldnames = ['Word', 'Sentiment', 'POS', 'Word Senses']
            try:
                with open(output_file, mode='r', encoding='utf-8') as
file:
                    processed_rows = list(csv.DictReader(file))
            except FileNotFoundError:
                processed_rows = []

            processed_words = {row['Word'] for row in processed_rows}

            with open(output_file, mode='w', encoding='utf-8',
newline='') as file:
                writer = csv.DictWriter(file, fieldnames=fieldnames)

```

```

        writer.writeheader()

        writer.writerows(processed_rows) # Write already
processed rows

    for row in rows:
        if row['Word'] in processed_words:
            continue

        word = row['Word']
        sentiment = row['Sentiment']
        pos = row['POS']
        sense = row['Word Senses']

        print(f"\nProcessing word: {word}")
        if sentiment == "N/A":
            sentiment = DataOptions.select_sentiment()
        if pos == "N/A":
            pos = DataOptions.select_pos()
        if sense == "N/A":
            sense = input(f"Provide word senses for
'{word}' (separate by semicolon): ")

        updated_row = {"Word": word, "Sentiment":
sentiment, "POS": pos, "Word Senses": sense}
        writer.writerow(updated_row)
        print(f"Progress saved for '{word}'.")

```

1.3.12 modify_column.py

```

import csv
from data_options import DataOptions

class ModifyColumn:
    @staticmethod
    def modify_column_for_word(input_file, output_file):
        """
        Allows user to modify a specific column for a given word.
        """
        with open(input_file, mode='r', encoding='utf-8') as file:
            reader = csv.DictReader(file)
            rows = list(reader)

```

```

        fieldnames = ['Word', 'Sentiment', 'POS', 'Word Senses']

        with open(output_file, mode='w', encoding='utf-8',
newline='') as file:
            writer = csv.DictWriter(file, fieldnames=fieldnames)
            writer.writeheader()

            for row in rows:
                word = row['Word']
                print(f"\nCurrent data for word '{word}':
Sentiment: {row['Sentiment']}, POS: {row['POS']}, Senses:
{row['Word Senses']}")
                modify = input(f"Do you want to modify '{word}'?
(yes/no): ").strip().lower()

                if modify == 'yes':
                    print("Available columns: Sentiment, POS, Word
Senses")
                    column = input("Enter column to modify:
").strip().lower()

                    if column == 'sentiment':
                        row['Sentiment'] =
DataOptions.select_sentiment()
                    elif column == 'pos':
                        row['POS'] = DataOptions.select_pos()
                    elif column == 'word senses':
                        row['Word Senses'] = input("Enter new word
senses: ").strip()
                    else:
                        print("Invalid column. Skipping
modification.")

                writer.writerow(row)

            print(f"Modifications saved to {output_file}")

```

1.3.13 data_options.py

```

class DataOptions:
    SENTIMENT_OPTIONS = {

```

```

        "1": "positive",
        "2": "neutral",
        "3": "negative"
    }

    POS_OPTIONS = {
        "1": "kata nama",
        "2": "kata kerja",
        "3": "kata adjektif",
        "4": "kata ganti nama",
        "5": "kata bilangan",
        "6": "kata adverba",
        "7": "kata seru",
        "8": "kata hubung",
        "9": "kata sendi nama",
        "10": "kata nafi",
        "11": "kata tanya",
        "12": "kata perintah"
    }

    @staticmethod
    def select_sentiment():
        print("\nSelect sentiment:")
        for key, value in DataOptions.SENTIMENT_OPTIONS.items():
            print(f"{key}. {value.capitalize()}")
        choice = input("Enter your choice (1/2/3): ").strip()
        return DataOptions.SENTIMENT_OPTIONS.get(choice, "N/A")

    @staticmethod
    def select_pos():
        print("\nSelect POS (Part of Speech):")
        for key, value in DataOptions.POS_OPTIONS.items():
            print(f"{key}. {value.capitalize()}")
        choice = input("Enter your choice (1-12): ").strip()
        return DataOptions.POS_OPTIONS.get(choice, "N/A")

```

1.3.14 lexicon_functions.py

```
import csv
```

```

class LexiconProcessor:
    def __init__(self, input_file, output_file):
        self.input_file = input_file
        self.output_file = output_file

    def extract_data(self, word_senses, key, word_list):
        if word_senses and key in word_senses:
            start_index = word_senses.find(key) + len(key)
            extracted_word =
word_senses[start_index:].split(";")[0].strip()
            if extracted_word in word_list: # Validate if the
word exists in the Word list
                return extracted_word
            return "N/A" # Return "N/A" if no valid synonym/antonym
is found

    def process_and_update_rows(self):
        # Read the CSV file and process rows
        with open(self.input_file, mode='r', encoding='utf-8') as
file:
            reader = csv.DictReader(file)
            rows = list(reader)
            word_list = [row["Word"] for row in rows] # Extract
the Word list

            # Process and update rows
            for row in rows:
                word_senses = row.get("Word Senses", "")
                row["Synonym"] = self.extract_data(word_senses,
"Sinonim: ", word_list)
                row["Antonym"] = self.extract_data(word_senses,
"Antonim: ", word_list)

            # Write the cleaned data to a new CSV file
            with open(self.output_file, mode='w', encoding='utf-8',
newline='') as file:
                fieldnames = list(rows[0].keys()) # Include only
unique columns
                writer = csv.DictWriter(file, fieldnames=fieldnames)
                writer.writeheader()

```

```

        writer.writerows(rows)

    print(f"Cleaned and updated data written to
{self.output_file}")

    def build_synonym_antonym_mappings(self, data):
        synonym_mapping = {}
        antonym_mapping = {}

        for row in data:
            word = row['Word'].strip()
            synonyms = row['Synonym'].strip() if 'Synonym' in row
and row['Synonym'].strip() != "N/A" else ""
            antonyms = row['Antonym'].strip() if 'Antonym' in row
and row['Antonym'].strip() != "N/A" else ""

            # Build synonym mapping
            if synonyms:
                for synonym in synonyms.split(";"):
                    synonym = synonym.strip()
                    if synonym: # Ensure non-empty synonyms
                        synonym_mapping.setdefault(word,
set()).add(synonym)
                        synonym_mapping.setdefault(synonym,
set()).add(word)

            # Build antonym mapping
            if antonyms:
                for antonym in antonyms.split(";"):
                    antonym = antonym.strip()
                    if antonym: # Ensure non-empty antonyms
                        antonym_mapping.setdefault(word,
set()).add(antonym)
                        antonym_mapping.setdefault(antonym,
set()).add(word)

        return synonym_mapping, antonym_mapping

    def update_data_with_mappings(self, data, synonym_mapping,
antonym_mapping):
        for row in data:

```

```

        word = row['Word'].strip()

        # Update synonyms
        if word in synonym_mapping:
            row['Synonym'] =
";".join(sorted(synonym_mapping[word]))
        else:
            row['Synonym'] = "N/A"

        # Update antonyms
        if word in antonym_mapping:
            row['Antonym'] =
";".join(sorted(antonym_mapping[word]))
        else:
            row['Antonym'] = "N/A"

    return data

def validate_and_display_stats(self, input_file):
    total_records = 0
    synonyms_found = 0
    antonyms_found = 0
    both_found = 0
    neither_found = 0

    with open(input_file, mode='r', encoding='utf-8') as file:
        reader = csv.DictReader(file)
        for row in reader:
            total_records += 1
            synonym = row.get("Synonym", "N/A")
            antonym = row.get("Antonym", "N/A")

            if synonym != "N/A":
                synonyms_found += 1
            if antonym != "N/A":
                antonyms_found += 1
            if synonym != "N/A" and antonym != "N/A":
                both_found += 1
            if synonym == "N/A" and antonym == "N/A":
                neither_found += 1

```

```
# Display the statistics
print("Data Validation and Insights:")
print(f"Total Records: {total_records}")
print(f"Records with Synonyms Found: {synonyms_found}")
print(f"Records with Antonyms Found: {antonyms_found}")
print(f"Records with Both Synonyms and Antonyms Found:
{both_found}")
print(f"Records with Neither Synonyms nor Antonyms:
{neither_found}")
```

1.4. Data Quantity

Total Article Data File : 19

Total Record of words : 2425 unique words

Records with Synonyms Found: 16

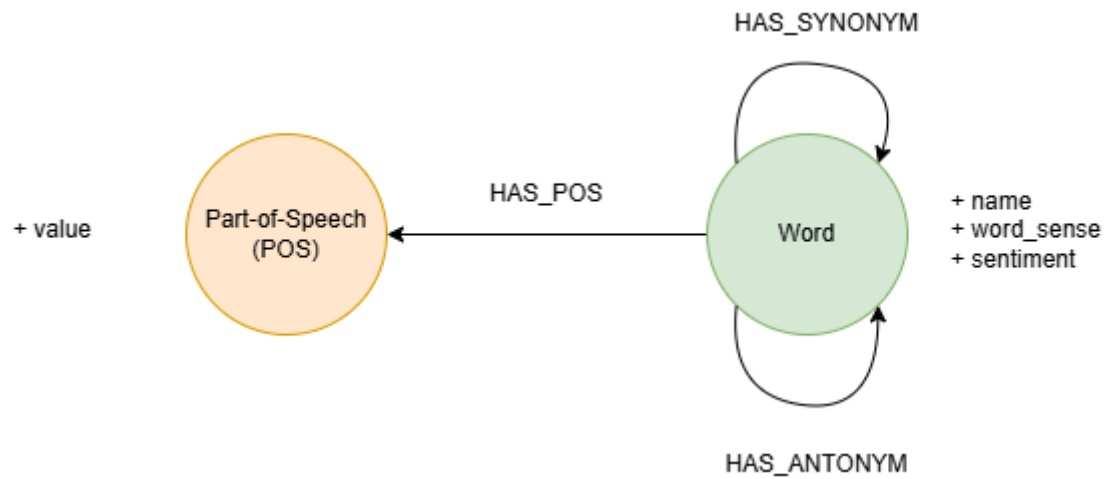
Records with Antonyms Found: 10

Records with Both Synonyms and Antonyms Found: 0

Records with Neither Synonyms nor Antonyms: 2399

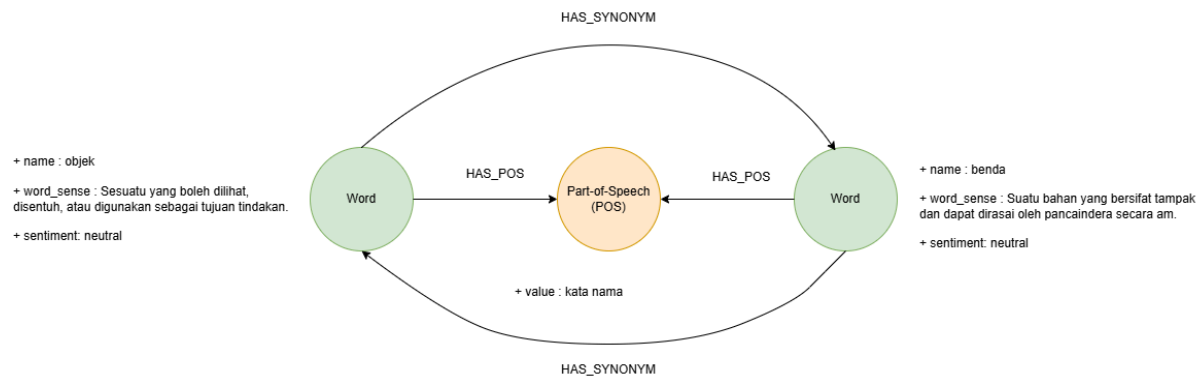
2. Lexicon Creation

2.1. Data Model Design



2.2. Example of an Entry in the Data Store





2.3. List of Python classes and Jupyter notebook(s) for lexicon:

Name of Python class(es) / notebooks	Author
neo4j_connector.py	Hea Hui Min
word_finder.py	Hea Hui Min

2.4. Code for Python Classes

2.4.1 neo4j_connector.py

```
from neo4j import GraphDatabase

# Function to create a connection to Neo4j
def create_neo4j_driver():

    neo4j_url = "neo4j+s://fae9ddfb.databases.neo4j.io"
    neo4j_username = "neo4j"
    neo4j_password = "lmggfQIHZtqYTo9LZeTKYA3QVf5XpSXz5p9VYUeQr1E"

    # Create a driver to interact with Neo4j
    driver = GraphDatabase.driver(neo4j_url, auth=(neo4j_username,
neo4j_password))
    return driver
```

```

# Function to load data into Neo4j from the DataFrame
def load_data_to_neo4j(df, driver):
    with driver.session() as session:
        for data_row in df.collect():
            word_name = data_row['Word']
            sentiment = data_row['Sentiment']
            pos_values = data_row['POS']
            word_senses = data_row['Word Senses']
            synonyms_values = data_row['Synonym']
            antonyms_values = data_row['Antonym']

            # Handle POS
            if pos_values:
                pos_list = pos_values.split(';')
            else:
                pos_list = ['Unknown']

            # Handle Word Senses
            if not word_senses:
                word_senses = 'Unknown'

            # Handle Synonyms
            if synonyms_values:
                synonyms = [synonym for synonym in
synonyms_values.split(';') if synonym.strip().lower() != "n/a"]
            else:
                synonyms = None

            # Handle Antonyms
            if antonyms_values:
                antonyms = [antonym for antonym in
antonyms_values.split(';') if antonym.strip().lower() != "n/a"]
            else:
                antonyms = None

            # Start a transaction in Neo4j
            with session.begin_transaction() as tx:

                # Create Word node with sentiment and word senses

```

```

as properties
    query = """
        MERGE (w:Word {name: $name})
        SET w.sentiment = $sentiment, w.word_senses =
$word_senses
    """
    params = {
        'name': word_name.strip(), # Ensure no extra
spaces around the name
        'sentiment': sentiment,
        'word_senses': word_senses # Use the value as
a single string
    }

    # Execute the query
    tx.run(query, params)

    # For each POS value
    for pos_value in pos_list:
        tx.run("""
            MERGE (p:POS {value: $value})
            MERGE (w:Word {name: $word_name})
            MERGE (w)-[:HAS_POS]->(p)
            ON CREATE SET p.value = $value
            """, value=pos_value.strip(),
word_name=word_name.strip())

        # If synonyms exist, create relationships between
Word and other Word nodes for synonyms
        if synonyms:
            for synonym in synonyms:
                tx.run("""
                    MERGE (w_synonym:Word {name:
$synonym})

                    MERGE (w:Word {name: $word_name})
                    MERGE (w)-[:HAS_SYNONYM]->(w_synonym)
                    MERGE (w_synonym)-[:HAS_SYNONYM]->(w)
                    """, synonym=synonym.strip(),
word_name=word_name.strip())

        # If antonyms exist, create relationships between

```

Word and other Word nodes for antonyms

```
    if antonyms:
        for antonym in antonyms:
            tx.run("""
                MERGE (w_antonym:Word {name:
$antonym})

                MERGE (w:Word {name: $word_name})
                MERGE (w)-[:HAS_ANTONYM]->(w_antonym)
                MERGE (w_antonym)-[:HAS_ANTONYM]->(w)
            """, antonym=antonym.strip(),
word_name=word_name.strip())
```

2.4.2 word_finder.py

```
class WordFinder:
    def __init__(self, driver):
        self.driver = driver

    def find_word_info(self, word):
        with self.driver.session() as session:
            result = session.read_transaction(self._find_word,
word)

            if result:
                # If the word is found, return its details,
including synonyms and antonyms
                sentiment, pos, word_sense, synonyms, antonyms =
result

                print(f"Word: {word}")
                print(f"Sentiment: {sentiment}")
                print(f"Part of Speech (POS): {pos}")
                print(f"Word Sense: {word_sense}")
                print(f"Synonyms: {'', '.join(synonyms) if synonyms
else 'None'}")
                print(f"Antonyms: {'', '.join(antonyms) if antonyms
else 'None'}")

            else:
                # If the word is not found
                print(f"Word '{word}' not found in the database.")

# Staticmethod is methods that perform simple tasks that don't
require access to any instance-specific data
@staticmethod
def _find_word(tx, word):

    query = (
        "MATCH (w:Word {name: $word}) "
        "OPTIONAL MATCH (w)-[:HAS_POS]->(p:POS) "
        "OPTIONAL MATCH (w)-[:HAS_SYNONYM]->(synonym:Word) "
        "OPTIONAL MATCH (w)-[:HAS_ANTONYM]->(antonym:Word) "
        "RETURN w.sentiment AS sentiment, "
        "COLLECT(DISTINCT p.value) AS pos, "
```

```
        "w.word_senses AS word_sense, "  
        "COLLECT(DISTINCT synonym.name) AS synonyms, "  
        "COLLECT(DISTINCT antonym.name) AS antonyms"  
    )  
    result = tx.run(query, word=word)  
  
    record = result.single()  
    if record:  
        return (  
            record['sentiment'],  
            record['pos'],  
            record['word_sense'],  
            record['synonyms'],  
            record['antonyms']  
        )  
    return None  
  
def close(self):  
    self.driver.close()
```

3. Lexicon Enrichment

Sentiment Analysis [Sentiment Labelling] - Hea Hui Min

3.1. List of Enrichment Carried Out

Word	Sentiment(Malaya)	Word	Sentiment(Manual)
pengkelassan	neutral	pengkelassan	neutral
penglihatan	neutral	penglihatan	neutral
pengosmokawalaturan	neutral	pengosmokawalaturan	neutral
pengubahsuaian	neutral	pengubahsuaian	neutral
penguin	neutral	penguin	neutral
pensenyawaan	positive	pensenyawaan	neutral
penspesiesan	neutral	penspesiesan	neutral
pentakrifan	neutral	pentakrifan	neutral
penternak	negative	penternak	neutral
penting	neutral	penting	positive
penumpuan	neutral	penumpuan	neutral
penurunan	neutral	penurunan	negative
penyakit	negative	penyakit	negative
penyaring	negative	penyaring	neutral
penyelidikan	neutral	penyelidikan	neutral
penyengat	negative	penyengat	negative
penyepit	neutral	penyepit	neutral
penyerapan	negative	penyerapan	neutral
penyesuaian	neutral	penyesuaian	neutral
penyingkiran	negative	penyingkiran	neutral

only showing top 20 rows

Total Word Records: 2425

Word	Sentiment (Manual)	Sentiment (Malaya)	Manual_Sentiment_Index	Malaya_Sentiment_Index
baylisascaris	negative	neutral	2.0	0.0
berkoloni	positive	neutral	1.0	0.0
direka	neutral	negative	0.0	1.0
hayy	neutral	positive	0.0	2.0
hingga	neutral	positive	0.0	2.0
kemarau	negative	neutral	2.0	0.0
menyusut	neutral	negative	0.0	1.0
najis	negative	neutral	2.0	0.0
pemangsa	negative	neutral	2.0	0.0
sinaran	positive	neutral	1.0	0.0
tertanam	positive	neutral	1.0	0.0
whipworms	negative	neutral	2.0	0.0
kekal	positive	neutral	1.0	0.0
kepah	neutral	negative	0.0	1.0
kerang	neutral	negative	0.0	1.0
mikromh	neutral	negative	0.0	1.0
nemertea	neutral	positive	0.0	2.0
pengarangnya	neutral	positive	0.0	2.0
pertanian	neutral	positive	0.0	2.0
protostomes	neutral	negative	0.0	1.0

only showing top 20 rows

Incorrect Predictions: 1062

3.2. List of Python classes and Jupyter notebook(s) for lexicon enrichment:

Name of Python class(es) / notebooks	Author
Gm4-5_HeaHuiMin.ipynb	Hea Hui Min

4. Lexicon Analysis and Evaluation

4.1. Results of Analysis

4.1.1 Word Frequency Analysis (Point 1 & 2) - Cason Soh Kai Sheng

Token counts:

+-----+-----+	
token	count
+-----+-----+	
hingga	19
sedar	2
jinak	1
pemangsa	7
tapak	2
pertanian	9
taksonomi	20
simbol	2
alif	2
terdiri	30
antara	62
ekologi	13
pesisir	3
peraduan	1
dikenali	37
memulakan	4
tipu	2
hangus	1
menghidupkan	2
jantan	3
+-----+-----+	

only showing top 20 rows

Most common words:

+-----+-----+	
token	count
+-----+-----+	
atau	1721
yang	1497
sesuatu	573

only showing top 3 rows

Least common words:

+-----+-----+	
token	count
+-----+-----+	
peraduan	1
ascariasis	1
hangus	1

only showing top 3 rows

```

hingga: 19
sedar: 2
jinak: 1
pemangsa: 7
tapak: 2
pertanian: 9
taksonomi: 20
simbol: 2
alif: 2
terdiri: 30
antara: 62
ekologi: 13
pesisir: 3
peraduan: 1
dikenali: 37
memulakan: 4
tipu: 2
hangus: 1
menghidupkan: 2
jantan: 3
tekanan: 4
perlahan: 8
tumbuhan: 103
bermaksud: 5
sesuai: 6
rotan: 1
transitif: 17
seseorang: 82
panjang: 39
istilah: 9
ascariasis: 1
ilmu: 27
dilakukan: 25
sebab: 9

```

To show all rows >

Below is to find a word's occurrence.

```

Enter a word to find its occurrence (or type 'exit' to quit): ekologi
The word 'ekologi' occurs 13 time(s).
Enter a word to find its occurrence (or type 'exit' to quit): name
The word 'name' does not exist in the dataset.
Enter a word to find its occurrence (or type 'exit' to quit): quit
The word 'quit' does not exist in the dataset.
Enter a word to find its occurrence (or type 'exit' to quit): exit
Exiting the program.

```

Below is to find the words by entering the number of occurrences.

```

Enter a number of occurrences to find words (or type 'exit' to quit): 50
Words with 50 occurrence(s):
- manusia
Enter a number of occurrences to find words (or type 'exit' to quit): 20
Words with 20 occurrence(s):
- taksonomi
- laba
- arthropoda
- cahaya
- dinosaur
- tahun
- baik
- kehidupan
- lagi
Enter a number of occurrences to find words (or type 'exit' to quit): exit
Exiting the program.

```

4.1.2 Word Frequency Analysis (Point 3) - Ethel Ng Yi Yan

Words with part of speech 'kata ganti nama': Words with part of speech 'kata bilangan':

Word	Part of Speech
anda	kata ganti nama
antaranya	kata ganti nama
apa-apa	kata ganti nama
apapun	kata ganti nama
asalnya	kata ganti nama
atasnya	kata ganti nama
awalnya	kata ganti nama
badannya	kata ganti nama
bangkainya	kata ganti nama
banyaknya	kata ganti nama
terluasnya	kata ganti nama
tiadanya	kata ganti nama
tua	kata ganti nama
tubuhnya	kata ganti nama
zuriatnya	kata ganti nama

Total number of 'kata ganti nama': 53

Word	Part of Speech
banyak	kata bilangan
batang	kata bilangan
batu	kata bilangan
beberapa	kata bilangan
beratus	kata bilangan
berbagai-bagai	kata bilangan
berbilang	kata bilangan
beribu	kata bilangan
biji	kata bilangan
bilangan	kata bilangan
buah	kata bilangan
buku	kata bilangan
sepunya	kata bilangan
setiap	kata bilangan
suatu	kata bilangan
tiga	kata bilangan
tujuh	kata bilangan

Total number of 'kata bilangan': 52

Words with part of speech 'kata hubung':

Word	Part of Speech
agar	kata hubung
and	kata hubung
antara	kata hubung
ataupun	kata hubung
bagaimanapun	kata hubung
bahawa	kata hubung
bahkan	kata hubung
selama	kata hubung
sementara	kata hubung
serta	kata hubung
sungguhpun	kata hubung
supaya	kata hubung
walaupun	kata hubung

Total number of 'kata hubung': 39

Words with part of speech 'kata adjektif':

Word	Part of Speech
abad	kata adjektif
aboral	kata adjektif
adil	kata adjektif
akhir	kata adjektif
aktif	kata adjektif
akuatik	kata adjektif
akustik	kata adjektif
amat	kata adjektif
ampai-ampai	kata adjektif
usus	kata adjektif
utama	kata adjektif
vascular	kata adjektif
vaskular	kata adjektif
vernakular	kata adjektif

Total number of 'kata adjektif': 359

Words with part of speech 'kata nama':

Word	Part of Speech
abdomen	kata nama
abdomennya	kata nama
abjad	kata nama
abu	kata nama
acanthocephala	kata nama
acari	kata nama
acariformes	kata nama
acoelomorpha	kata nama
actinopterygii	kata nama
ada	kata nama
adanya	kata nama
adaptasi	kata nama
adenophorea	kata nama
zarah-zarah	kata nama
zat	kata nama
zigot	kata nama
zooid	kata nama
zooxanthellae	kata nama

Total number of 'kata nama': 1365

Words with part of speech 'kata kerja':

Word	Part of Speech
abad	kata kerja
ada	kata kerja
agak	kata kerja
akan	kata kerja
akhir	kata kerja
alam	kata kerja
alih	kata kerja
ampai	kata kerja
angkat	kata kerja
arab	kata kerja
usaha	kata kerja
usia	kata kerja
walaupun	kata kerja
wujud	kata kerja
zon	kata kerja

Total number of 'kata kerja': 561

Words with part of speech 'kata sendi nama':

Word	Part of Speech
amnya	kata sendi nama
apabila	kata sendi nama
berserta	kata sendi nama
daripadanya	kata sendi nama
demi	kata sendi nama
hinggalah	kata sendi nama
juga	kata sendi nama
kecuali	kata sendi nama
kemana-mana	kata sendi nama
kepada	kata sendi nama
lima	kata sendi nama
samping	kata sendi nama
sebelum	kata sendi nama
selepas	kata sendi nama
sebelumnya	kata sendi nama
tentang	kata sendi nama
terhadap	kata sendi nama
terhubung	kata sendi nama

Total number of 'kata sendi nama': 32

Words with part of speech 'kata seru':

Word	Part of Speech
babi	kata seru
bulatlah	kata seru
gas	kata seru

Total number of 'kata seru': 3

Words with part of speech 'kata tanya':

Word	Part of Speech
ampai	kata tanya
baja	kata tanya
mana	kata tanya

Total number of 'kata tanya': 3

Words with part of speech 'kata nafi':

Word	Part of Speech
bukan	kata nafi
bukannya	kata nafi

Words with part of speech 'kata adverba':

+-----+-----+	
Word	Part of Speech
+-----+-----+	
adakalanya	kata adverba
awal-awal	kata adverba
barangkali	kata adverba
baru-baru	kata adverba
bawah	kata adverba
belum	kata adverba
tengah-tengah	kata adverba
terutama	kata adverba
terutamanya	kata adverba
umumnya	kata adverba
yakni	kata adverba
zaman	kata adverba
+-----+-----+	

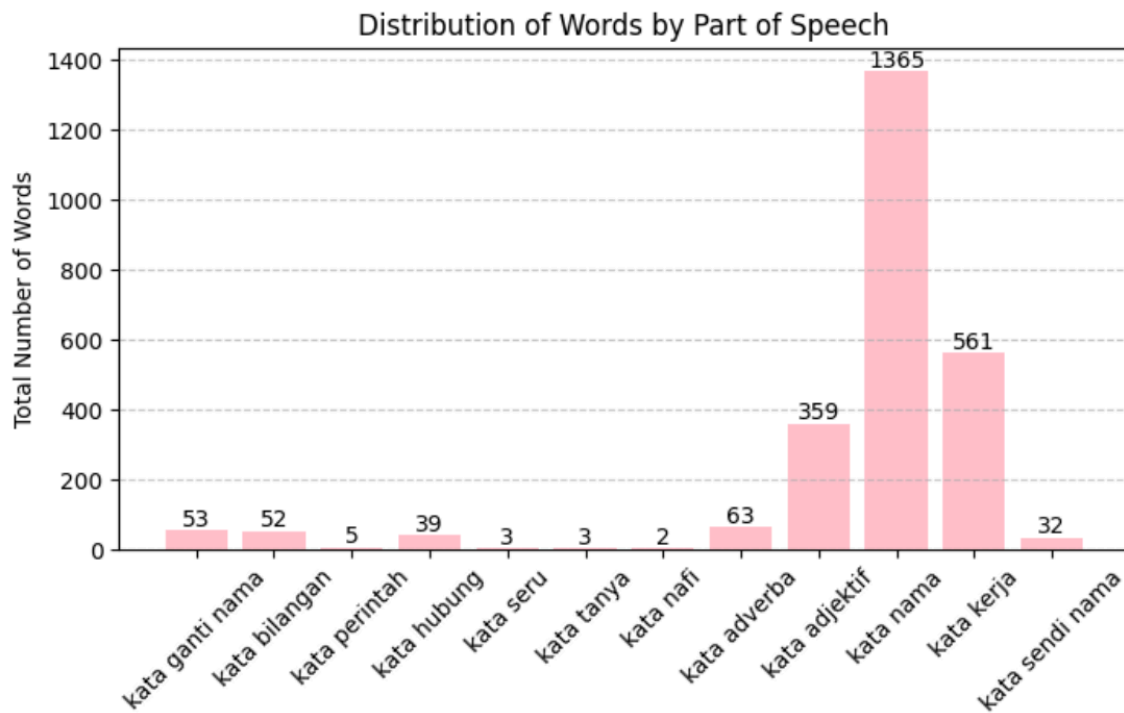
Total number of 'kata adverba': 63

^ To show all the words separately by categorising them using 11 different part-of-speech

4.1.3 Part-Of-Speech Distribution- Ethel Ng Yi Yan

POS and Total Number of Words:

Part of Speech	Total Number of Words
kata nama	1365
kata kerja	561
kata adjektif	359
kata adverba	63
kata ganti nama	53
kata bilangan	52
kata hubung	39
kata sendi nama	32
kata perintah	5
kata seru	3
kata tanya	3
kata nafi	2

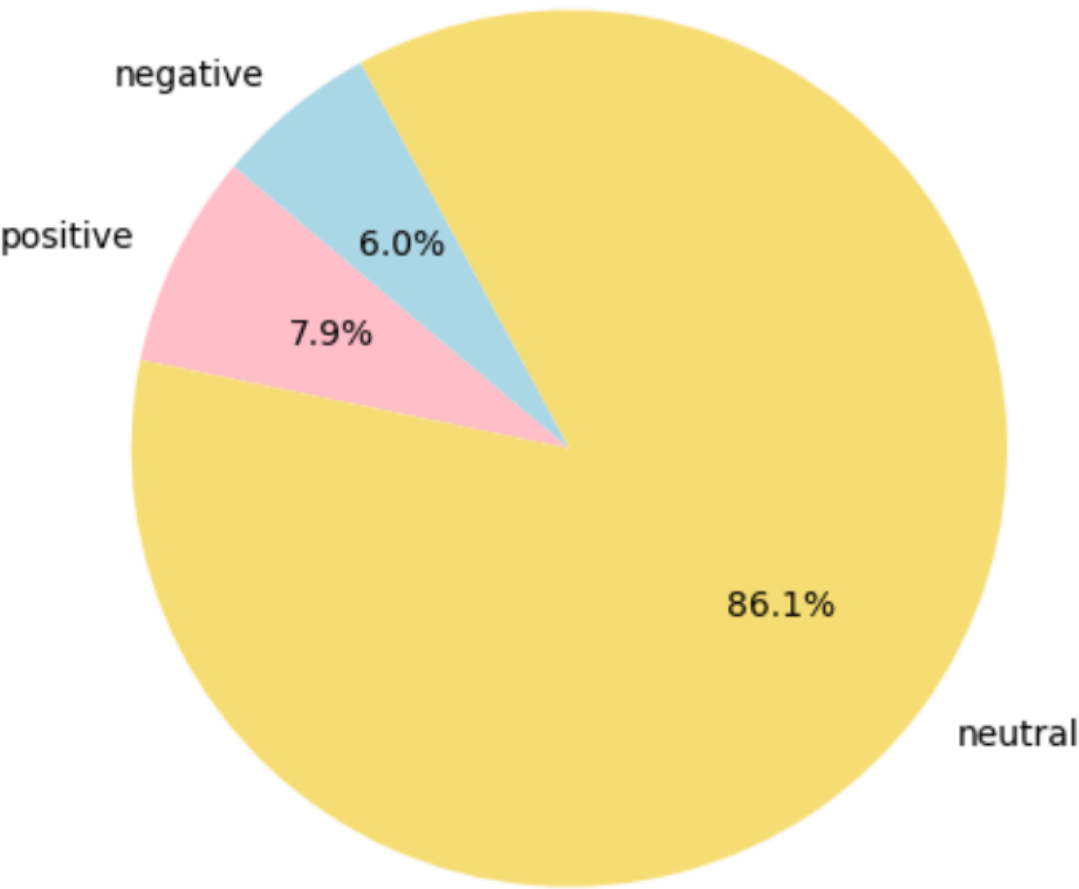


The diagram above shows the distribution of words by part of speech of our lexicon. “kata nama” which is known as a noun has the most frequency which has a number of 1365 where “kata nafi” has the least number of words which consists of only 2 records. Therefore, we can say that the lexicon is skewed towards “kata nama”.

4.1.4 Sentiment Distribution - Ethel Ng Yi Yan

Sentiment	Word Count	Proportion	Percentage
positive	191	0.0788	7.88%
neutral	2089	0.8614	86.14%
negative	145	0.0598	5.98%

Proportion of Words by Sentiment



4.1.5 Morphological Analysis - Gan She Ming

Word	Prefix	Suffix	Stem
beradik	ber	None	adik
berahang	ber	None	ahang
berair	ber	None	air
berambut	ber	None	ambut
beranak	ber	None	anak
beranggota	ber	None	anggota
beransur-ansur	ber	None	ansur-ansur
berasal	ber	None	asal
berasaskan	ber	kan	asas
berasingan	ber	an	asing

... more row

Most Common Prefixes:

Prefix	Count
me	223
ber	154
di	144
se	131
ke	103
pe	91
ter	76
per	56
te	35

< Find most common prefixes

Most Common Suffixes:

Suffix	Count
an	284
kan	169
nya	128
si	37
ti	25
lah	18
logi	6
ku	4
isme	3
mu	1
isasi	1

< Find most common suffixes

Number of words with only prefix: 582

Number of words with only suffix: 245

Number of words with both prefix and suffix: 431

Number of words with neither prefix nor suffix: 1167

Probabilities:

Probability of words with only prefix: 0.2400 (24.00%)

Probability of words with only suffix: 0.1010 (10.10%)

Probability of words with both prefix and suffix: 0.1777 (17.77%)

Probability of words with neither prefix nor suffix: 0.4812 (48.12%)

^ To show number of words and its probabilities with only/both/neither prefix and suffix

Prefix-Suffix Combinations and Counts (Sorted by Probability):

Prefix-Suffix Combination	Count	Probability (%)
me-kan	82 (3.38%)	0.0338
di-kan	59 (2.43%)	0.0243
pe-an	52 (2.14%)	0.0214
ke-an	41 (1.69%)	0.0169
per-an	37 (1.53%)	0.0153
ber-an	23 (0.95%)	0.0095
se-nya	20 (0.82%)	0.0082
me-nya	11 (0.45%)	0.0045
ber-si	10 (0.41%)	0.0041
ke-nya	10 (0.41%)	0.0041
se-an	9 (0.37%)	0.0037
di-ti	7 (0.29%)	0.0029
ter-nya	6 (0.25%)	0.0025
me-si	5 (0.21%)	0.0021
me-an	5 (0.21%)	0.0021
me-ti	5 (0.21%)	0.0021
pe-kan	5 (0.21%)	0.0021
di-an	4 (0.16%)	0.0016
ke-kan	4 (0.16%)	0.0016
ber-kan	3 (0.12%)	0.0012
se-kan	3 (0.12%)	0.0012
se-lah	3 (0.12%)	0.0012
te-nya	3 (0.12%)	0.0012
ber-nya	2 (0.08%)	0.0008
ber-ti	2 (0.08%)	0.0008
di-nya	2 (0.08%)	0.0008
me-isme	2 (0.08%)	0.0008
pe-nya	2 (0.08%)	0.0008
per-nya	2 (0.08%)	0.0008
ter-an	2 (0.08%)	0.0008
ber-ku	1 (0.04%)	0.0004
di-lah	1 (0.04%)	0.0004
per-kan	1 (0.04%)	0.0004
se-ti	1 (0.04%)	0.0004
te-an	1 (0.04%)	0.0004
te-lah	1 (0.04%)	0.0004
te-ti	1 (0.04%)	0.0004
ter-ti	1 (0.04%)	0.0004
ter-si	1 (0.04%)	0.0004
ter-lah	1 (0.04%)	0.0004

^ To show the probabilities of all prefix-suffix combinations

4.1.6 Word Length and Complexity Analysis - Gan She Ming

Word Length	Frequency
8	351
7	334
5	302
9	292
6	260
10	247
11	166
4	163
12	131
13	59
14	37
3	30
15	24
16	16
17	7
18	3
19	2
22	1

Average Word Length: 8.077525773195877

Longest word: karangsetidak-tidaknya with length 22

Shortest word: dia with length 3

length	frequency	probability	percentage
8	351	0.1447	14.47
7	334	0.1377	13.77
5	302	0.1245	12.45
9	292	0.1204	12.04
6	260	0.1072	10.72
10	247	0.1019	10.19
11	166	0.0685	6.85
4	163	0.0672	6.72
12	131	0.054	5.4
13	59	0.0243	2.43
14	37	0.0153	1.53
3	30	0.0124	1.24
15	24	0.0099	0.99
16	16	0.0066	0.66
17	7	0.0029	0.29
18	3	0.0012	0.12
19	2	8.0E-4	0.08
22	1	4.0E-4	0.04

4.2. Results of Intrinsic Evaluation

3.1 Sentiment Labelling Accuracy

Word	Sentiment (Manual)	Sentiment (Malaya)	Manual_Sentiment_Index	Malaya_Sentiment_Index
baylisascaris	negative	neutral	2.0	0.0
berkoloni	positive	neutral	1.0	0.0
direka	neutral	negative	0.0	1.0
generasi	neutral	neutral	0.0	0.0
genetik	neutral	neutral	0.0	0.0
hayy	neutral	positive	0.0	2.0
heksapod	neutral	neutral	0.0	0.0
hingga	neutral	positive	0.0	2.0
kemarau	negative	neutral	2.0	0.0
menyusut	neutral	negative	0.0	1.0
najis	negative	neutral	2.0	0.0
pemangsa	negative	neutral	2.0	0.0
sinaran	positive	neutral	1.0	0.0
tapak	neutral	neutral	0.0	0.0
tertanam	positive	neutral	1.0	0.0
tumbuh	neutral	neutral	0.0	0.0
whipworms	negative	neutral	2.0	0.0
berkait	neutral	neutral	0.0	0.0
cladistics	neutral	neutral	0.0	0.0
kekal	positive	neutral	1.0	0.0

only showing top 20 rows

Accuracy: 0.5620618556701031

Accuracy: 56.21%

Correct Predictions: 1363

Total Predictions: 2425

Incorrect Predictions: 1062

4.3. List of Python classes and Jupyter notebook(s) for lexicon Analysis and Evaluation:

Name of Python class(es) / notebooks	Author
GM4-5_EthelNgYiYan.ipynb	Ethel Ng Yi Yan
GM4-5_CasonSohKaiSheng.ipynb	Cason Soh Kai Sheng
GM4-5_GanSheMing.ipynb	Gan She Ming

5. Data Streaming

5.1. Kafka Streaming

Use case:	Can use the extracted paragraphs to build a real-time content summarization system, providing concise overviews for education purposes.
Kafka Topic:	article_topic
End user(s):	Educators and students

Screenshot of message content:

```
student@SSI:~$ source assignment-folder/asf-venv/bin/activate
(asf-venv) student@SSI:~$ python /home/student/assignment-folder/kafkaConsumer.py
Subscribed to topic 'article_topic'. Waiting for messages...

Received article:
Title: Binatang
Content: Haiwan (Jawi: هائوانcode: ms is deprecated serapan Arab: هائوان rumi: hayawāncode: ar is deprecated terbitan kata هائcode: ar is deprecated hayy "hidup"[1]), atau binatang (Jawi: بانڠتڠcode: ms is deprecated ) ialah sekelompok besar organisma eukariot multisel yang tergolong dalam alam Animalia atau Metazoa. Apabila berkembang, pelan badannya pada kesudahannya menjadi tetap, namun ada sesetengah spesies yang menjalankan proses metamorfosis di tengah-tengah riwayatnya. Kebanyakan ha...

Received article:
Title: Ikan
Content: Ikan (Jawi: ايكانcode: ms is deprecated ) ialah sejenis haiwan vertebrata akuatik yang tidak beramion dengan luaran yang mempunyai insang, sirip untuk renang, serta tengkorak keras, tetapi tidak mempunyai anggota badan dengan jejari. Ikan boleh dikelompokkan kepada ikan tanpa rahang yang lebih bawah kedua ukannya secara filogenetik, serta ikan berahang yang lebih biasa ditemui. Ikan berahang merangkumi kesemua spesies yang masih wujud pada masa ini seperti ikan berawan dan ikan bertulang, ser...

Received article:
Title: Burung
Content: Burung (Jawi: بورڠcode: ms is deprecated ) merupakan haiwan vertebrata bersayap, berkaki dua, berdarah panas dan bertelur dalam kelas Aves yang besar dan pelbagai terdapat di serata dunia, dari kawasan gurun sehinggalah di Kutub Utara, juga di kawasan hutan hujan Amazon, dan Greenland. Terdapat lebih daripada 8,600 spesies burung yang telahpun dikenal pasti yang dibahagikan kepada 27 order. Selain itu, terdapat banyak subspesies yang jika dikira berserta dengan spesies yang diketahui menga...

Received article:
Title: Reptilia
Content: Reptilia tergolong dalam benda hidup, kumpulan haiwan, mempunyai tulang belakang. Ciri-ciri haiwan reptilia adalah seperti berikut :- Contoh-contoh haiwan dari keluarga reptilia :- Reptilia merupakan haiwan berdarah sejuk yang dibahagi kepada empat order yang masih hidup. Reptilia boleh di dapati diseluruh dunia dari kawasan padang pasir yang kering, di pusat bandar, sehingga beratus meter di dalam laut. Bagaimanapun reptilia tidak terdapat di kawasan kutub dan puncak gunung. Disebabkan reptili...
```

List of Python classes for Kafka producer and consumer:

Name of Python class(es) / notebooks	Author
kafkaConsumer.py	Cason
kafkaProducer.py	Cason

5.2. Python Code

5.2.1 kafkaProducer

```
from kafka import KafkaProducer
import json

# Kafka Producer class setup
class KafkaProducerWrapper:
    def __init__(self, bootstrap_servers='localhost:9092'):
        try:
            self.producer = KafkaProducer(
                bootstrap_servers=bootstrap_servers,
                value_serializer=lambda v:
                    json.dumps(v).encode('utf-8') # Serialize data to JSON
            )
            print("Kafka Producer initialized successfully.")
        except Exception as e:
            print(f"Failed to initialize Kafka Producer: {e}")
            self.producer = None

    def send_message(self, topic, message):
        if not self.producer:
            print("Kafka Producer is not initialized. Cannot send message.")
            return False
        try:
            self.producer.send(topic, message)
            self.producer.flush()
            print(f"Message sent to Kafka topic '{topic}': {message}")
            return True
        except Exception as e:
            print(f"Failed to send message to Kafka: {e}")
            return False

    def close(self):
        if self.producer:
            self.producer.close()
            print("Kafka Producer closed.")
```

5.2.2 kafkaConsumer

```
from kafka import KafkaConsumer
import json

# Kafka Consumer setup
consumer = KafkaConsumer(
    'article_topic', # Topic to subscribe to
    bootstrap_servers='localhost:9092', # Kafka server address
    auto_offset_reset='earliest', # Start reading at the earliest
    message
    enable_auto_commit=True, # Commit offsets automatically
    group_id='wikipedia_consumer_group', # Consumer group name
    value_deserializer=lambda v: json.loads(v.decode('utf-8')) #
    Deserialize JSON messages
)

print("Subscribed to topic 'article_topic'. Waiting for
messages...")

# Consume messages from Kafka topic
try:
    for message in consumer:
        article = message.value
        print("\nReceived article:")
        print(f"Title: {article['title']}")
        print(f"Content: {article['content'][:500]}...\n") #
        Print first 200 characters of content
except KeyboardInterrupt:
    print("\nConsumer stopped.")
finally:
    consumer.close()
```

5.3. Spark Structured Streaming

Use case:	The lexicon that contains the words and their respective word sense, sentiment, part-of-speech, synonym and antonym will be transfer to our Neo4j database through spark structure streaming
Source:	CSV files
Sink:	Neo4j Graph Database
End user(s):	Data Analysts

Screenshot of aggregated data that will be streamed:

<pre> root -- Word: string (nullable = true) -- Sentiment: string (nullable = true) -- POS: string (nullable = true) -- Word Senses: string (nullable = true) -- Synonym: string (nullable = true) -- Antonym: string (nullable = true) </pre>						
<pre> +-----+-----+-----+-----+-----+-----+ Word Sentiment POS Word Senses Synonym Antonym +-----+-----+-----+-----+-----+-----+ abad neutral kata kerja;kata a... Tempoh masa selam... N/A N/A abdomen neutral kata nama (anatomi) Bahagia... N/A N/A abdomennya neutral kata nama bahagian tubuh an... N/A N/A abjad neutral kata nama gabungan angka da... N/A N/A aboral neutral kata adjektif sesuatu yang terl... N/A N/A +-----+-----+-----+-----+-----+-----+ </pre>						

List of Python classes / Jupyter notebook(s) for Structured Streaming:

Name of Python class(es) / notebooks	Author
neo4j_connector.py	Hea Hui Min

5.4. Python Code

```
from neo4j import GraphDatabase

# Function to create a connection to Neo4j
def create_neo4j_driver():

    neo4j_url = "neo4j+s://fae9ddfb.databases.neo4j.io"
    neo4j_username = "neo4j"
    neo4j_password = "lmggfQIHZtqYTo9LZeTKYA3QVf5XpSXz5p9VYUeQr1E"

    # Create a driver to interact with Neo4j
    driver = GraphDatabase.driver(neo4j_url, auth=(neo4j_username,
neo4j_password))
    return driver
```

```

# Function to load data into Neo4j from the DataFrame
def load_data_to_neo4j(df, driver):
    with driver.session() as session:
        for data_row in df.collect():
            word_name = data_row['Word']
            sentiment = data_row['Sentiment']
            pos_values = data_row['POS']
            word_senses = data_row['Word Senses']
            synonyms_values = data_row['Synonym']
            antonyms_values = data_row['Antonym']

            # Handle POS
            if pos_values:
                pos_list = pos_values.split(';')
            else:
                pos_list = ['Unknown']

            # Handle Word Senses
            if not word_senses:
                word_senses = 'Unknown'

            # Handle Synonyms
            if synonyms_values:
                synonyms = [synonym for synonym in
synonyms_values.split(';') if synonym.strip().lower() != "n/a"]
            else:
                synonyms = None

            # Handle Antonyms
            if antonyms_values:
                antonyms = [antonym for antonym in
antonyms_values.split(';') if antonym.strip().lower() != "n/a"]
            else:
                antonyms = None

            # Start a transaction in Neo4j
            with session.begin_transaction() as tx:

                # Create Word node with sentiment and word senses
as properties
                query = """

```

```

        MERGE (w:Word {name: $name})
        SET w.sentiment = $sentiment, w.word_senses =
$word_senses
    """
    params = {
        'name': word_name.strip(), # Ensure no extra
spaces around the name
        'sentiment': sentiment,
        'word_senses': word_senses # Use the value as
a single string
    }

    # Execute the query
    tx.run(query, params)

    # For each POS value
    for pos_value in pos_list:
        tx.run("""
            MERGE (p:POS {value: $value})
            MERGE (w:Word {name: $word_name})
            MERGE (w)-[:HAS_POS]->(p)
            ON CREATE SET p.value = $value
            """, value=pos_value.strip(),
word_name=word_name.strip())

    # If synonyms exist, create relationships between
Word and other Word nodes for synonyms
    if synonyms:
        for synonym in synonyms:
            tx.run("""
                MERGE (w_synonym:Word {name:
$synonym})

                MERGE (w:Word {name: $word_name})
                MERGE (w)-[:HAS_SYNONYM]->(w_synonym)
                MERGE (w_synonym)-[:HAS_SYNONYM]->(w)
                """, synonym=synonym.strip(),
word_name=word_name.strip())

    # If antonyms exist, create relationships between
Word and other Word nodes for antonyms
    if antonyms:

```

```
for antonym in antonyms:
    tx.run("""
        MERGE (w_antonym:Word {name:
$antonym})

        MERGE (w:Word {name: $word_name})
        MERGE (w)-[:HAS_ANTONYM]->(w_antonym)
        MERGE (w_antonym)-[:HAS_ANTONYM]->(w)
    """, antonym=antonym.strip(),
word_name=word_name.strip())
```